

Table des matières

1	Définition	1
1.1	Fonction booléenne	1
1.2	Forme normal algébrique et degré	2
1.3	Code de Reed-Muller	2
1.4	Non linéarité	2
1.5	Rayon de recouvrement	2
2	Problématique	3
2.1	Inégalité de ρ	3
2.1.1	Décomposition de Plotkin	3
3	Algo	3
3.1	Algo probabiliste	3

1 Définition

1.1 Fonction booléenne

Une fonction booléenne est une fonction qui prend en paramètre un vecteur binaire et qui renvoie 0 ou 1.

$$f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$$

Soit f une fonction booléenne en n variable on définit le poids de f comme :

$$\text{wt}(f) = \#\{f(x) = 1 \mid x \in \mathbb{F}_2^n\}$$

On peut représenter une fonction par son vecteur d'évaluation (table de vérité) comme :

$$(f(0), f(1), \dots, f(2^n - 1))$$

On définit la distance de hamming de deux fonctions booléenne f et g comme :

$$d_H = \#\{f(x) \neq g(x) \mid x \in \mathbb{F}_2^n\}$$

ou

$$\text{wt}(f + g)$$

Soit C un Code $\subset \mathbb{F}_2^{2^n}$ et f une fonction booléenne en n variable on définit sa distance au code comme :

$$d(f, C) = \min_{c \in C} d_H(f, c)$$

TABLE 1 – Tableau des valeur de $\rho(k, n)$

$k \backslash n$	3	4	5	6	7	8	9	10
1	2	6	12	28	56	120	242-244	496
2	1	2	6	18	40	88-96	196-216	400-460
3	0	1	2	8	20	56-60	111-156	194-372
4		0	1	2	8	26	58-86	≤ 242
5			0	1	2	10	28-36	≤ 122
6				0	1	2	10	≤ 46
7					0	1	2	12
8						0	1	2

1.2 Forme normal algébrique et degré

Toute fonctions booléenne peut s'écrire comme une somme de polynôme nommé ANF (algebraic normal form)

$$f(x) = \sum_{I \subseteq \{1, \dots, n\}} a_I \prod_{i \in I} x_i$$

On note le degré d'une fonction comme étant le degré maximal de ses termes.

1.3 Code de Reed-Muller

On définit le code de Reed-Muller en n variable et de degré k $\text{RM}(k, n)$ comme l'ensemble des fonctions booléennes de degré inférieur ou égale à k .

$$\{f \in \mathbb{F}_2^n \mid \deg(f) \leq k\}$$

1.4 Non linéarité

Soit un code $C \in \text{RM}(k, n)$ et une fonction $f \in \mathbb{F}_2^n$ on définit la non linéarité de degré k de f comme sa distance au code C autrement dit à sa distance de toutes les fonctions de degré au plus k .

$$\text{NL}_k(f) = d(f, C)$$

1.5 Rayon de recouvrement

On note $\rho(k, n)$ le rayon de recouvrement d'un code $C \in \text{RM}(k, n)$ la distance maximale entre ce dernier et l'ensemble des fonctions $f \in \mathbb{F}_2^{2^n}$

$$\rho(k, n) = \max_{f \in \mathbb{F}_2^{2^n}} d(f, C)$$

2 Problématique

On voudrait connaître le rayon de recouvrement de $\text{RM}(2, 8)$ et de $\text{RM}(3, 8)$

2.1 Inégalité de ρ

On sait que

$$\rho(k, n) \leq \rho(k, n-1) + \rho(k-1, n-1)$$

donc pour $\text{RM}(2, 8)$ on voudrait savoir si il existe une fonction f qui atteindrait cette borne supérieure tel que $\text{NL}_2(f) = 96$.

2.1.1 Décomposition de Plotkin

Toute fonction booléenne $f \in \mathbb{F}_2^{2^n}$ peut s'écrire sous la forme

$$f = x_n g + h$$

avec $g \in \text{RM}(k-1, n-1)$ et $h \in \text{RM}(k, n-1)$ si $x_n = 0, f = h$ et si $x_n = 1, f = g + h$ la table de vérité de toute fonction est donc de la forme

$$[h \mid h + g]$$

Ce qui veut dire que pour que $\text{NL}_2(f) = 96$ il faut que pour n'importe quelle fonction $t = x_8 v + u \in \text{RM}(2, 8)$

$$\text{wt}(t + f) \geq 96 \equiv \text{wt}(h + v) + \text{wt}(h + v + g + u) \geq 96$$

3 Algo

3.1 Algo probabiliste

Remarques :

- **BIT(A, pos)** renvoie le bit en position **pos** de l'élément ou du mot entier **A** (c.-à-d. $(A \gg \text{pos}) \& 1$).
- **XOR_BLOCK(dest, src)** signifie pour $k = 0 \dots \text{int_par_ligne} - 1$:
 $\text{dest}[\bar{k}] \leftarrow \text{dest}[k] \oplus \text{src}[k]$.
- Les indices et opérations sur bits suivent la même arithmétique que dans votre code C (division entière par 64 et modulo 64 pour localiser le mot/bit).

Algorithm 1 Random translate

Require: G : Une matrice génératrice d'un code $\text{RM}(k, n)$ de dimension $[K, N]$

f : Une fonction booléenne en n variable représentée par sa table de vérité de longueur N

```
1: for  $i \leftarrow 0$  to  $K - 1$  do
2:   repeat
3:      $pivot \leftarrow \text{rand}() \bmod N$ 
4:   until  $G[i][pivot] = 1$ 
5:   for  $j \leftarrow i$  to  $K - 1$  do
6:     if  $j \neq i$  and  $G[j][pivot] = 1$  then
7:        $G[j] \leftarrow G[j] + G[i]$ 
8:     end if
9:   end for
10:  if  $f[i] = 1$  then
11:     $f \leftarrow f + G[i]$ 
12:  end if
13: end for
14: return  $f$ 
```
